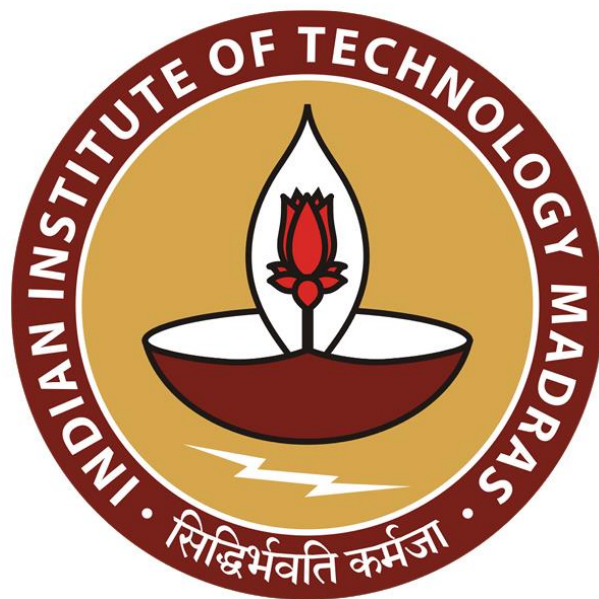




IIT Madras
ONLINE DEGREE

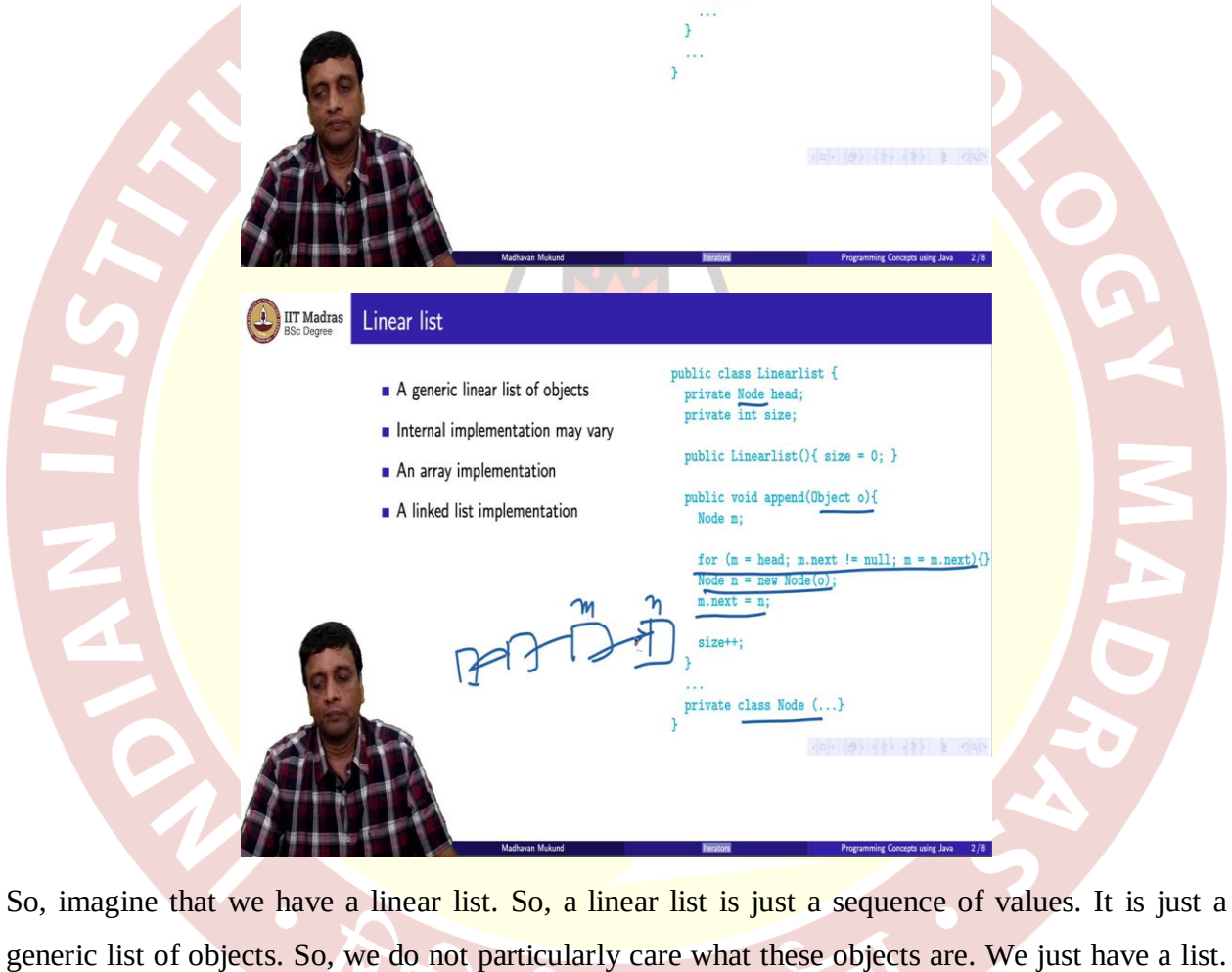
Programming Concepts using Java
Professor Madhavan Mukund,
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Iterators

(Refer Slide Time: 00:13)



So, the theme that we have been discussing is interaction with state. So, you have an object and you allow a certain interaction with it, but that interaction is governed by the state of the interaction. So, as the interaction proceeds the response changes. So, we saw how to do that, for example, with the database, where you wanted to make sure that the user is logged in. And after the user is logged in, only a limited number of queries can be executed. So, through a combination of private classes, private objects, and an interface, we can implement that. So, we are going to look at a very important example of this called an Iterator.

(Refer Slide Time: 00:48)



IIT Madras
BSc Degree

Linear list

- A generic linear list of objects
- Internal implementation may vary
- An array implementation

```
public class Linearlist {  
    // Array implementation  
    private int limit = 100;  
    private Object[] data = new Object[limit];  
    private int size; // Current size  
  
    public Linearlist(){ size = 0; }  
  
    public void append(Object o){  
        data[size] = o;  
        size++;  
        ...  
    }  
    ...  
}
```

Madhavan Mukund

Linear list

- A generic linear list of objects
- Internal implementation may vary
- An array implementation
- A linked list implementation

```
public class Linearlist {  
    private Node head;  
    private int size;  
  
    public Linearlist(){ size = 0; }  
  
    public void append(Object o){  
        Node n;  
  
        for (m = head; m.next != null; m = m.next){  
            Node n = new Node(o);  
            m.next = n;  
        }  
        size++;  
        ...  
    }  
    private class Node {...}  
}
```



Madhavan Mukund

So, imagine that we have a linear list. So, a linear list is just a sequence of values. It is just a generic list of objects. So, we do not particularly care what these objects are. We just have a list. And we could implement this in many different ways. So, for instance, we could have an array implementation, so we have an array called data, which is an array of object. And now we could have a linked list thing where we have a node. And this private class node will have two parts of it. It will have data part, which is of type object, and it will have a next type of node.

And so now if I want to append to such a thing, for example, if I want to add a list, I will first iterate, in this case, through this linked list by starting from the head and going through the next

until there is, I reached the last node which where, `m.next` is null, then I will create a new node and stick it at the end. So, I am basically walking down this list until I reach the node where there is no next and I create a new node `n`, and then I make this, so at this point, this is `m`, then I make `m.next` point to `n`. So, now I have two different implementations. So, remember, I had this array implementation and I have kind of a linked list implementation.

(Refer Slide Time: 02:10)

IIT Madras BSc Degree

Iteration

- Want a loop to run through all values in a linear list
- If the list is an array with public access, we write this
- For a linked list with public access, we could write this

```
int i;
for (i = 0; i < data.length; i++){
    ... // do something with data[i]
}

Node m;
for (m = head; m != null; m = m.next){
    ... // do something with m.data
}
```

Madhavan Mukund

Iteration

Programming Concepts using Java 3/8

IIT Madras BSc Degree

Iteration

- Want a loop to run through all values in a linear list
- If the list is an array with public access, we write this
- For a linked list with public access, we could write this
- We don't have public access ...
- ... and we don't know which implementation is in use!

```
int i;
for (i = 0; i < data.length; i++){
    ... // do something with data[i]
}

Node m;
for (m = head; m != null; m = m.next){
    ... // do something with m.data
}
```

Madhavan Mukund

Iteration

Programming Concepts using Java 3/8

Now, as a user of a linked list, it is quite natural to want to run through the elements in a list, so this is what we call iteration. We want a loop which says start from the first element in the list and go through all of them in sequence until the last one. So, how would we do this in this case?

So, that, remember that we had that variable called data, which is there inside the object. So, if it was public the data is an array, then we can just write a normal for loop. We can run from 0 to data.length, and do something with data[i].

On the other hand, if we know that that thing is linked list and we again have access to the head, we can start with the head. And if we can go through the elements in the list publicly, then we can do the same loop that we wrote for append. We can write a similar loop, which walks through all the elements in our list and does something with each data point.

But of course, from an encapsulation point of view, we would not typically have public access to the implementation. So, that is one important thing. So, this, both these things would require us to have public access to the contents of the linear list. And there is absolutely no reason in an object-oriented setting, why there should be public access to the internal data.

And more critically, because it is again a hidden encapsulated implementation, we do not even know whether it is this or that. We do not know whether it is an array or whether it is a linked list. So, which of these two strategies makes sense is not clear. So, we do not have access and we do not even know what type of underlying structure we are trying to iterate over, and yet we want to do this right. So, this is a very natural requirement. You give me a list. I want to run through the elements in the list. So, how do we do this?

(Refer Slide Time: 03:56)

IIT Madras BSc Degree

Iterators

- Need the following abstraction

```
Start at the beginning of the list; —
while (there is a next element){
  get the next element;
  do something with it
}
```

Madhavan Mukund

Iterators

Programming Concepts using Java 4/8

- Need the following abstraction

```
Start at the beginning of the list
while (there is a next element){
    get the next element;
    do something with it
}
```

- Encapsulate this functionality in an interface called `Iterator`

```
public interface Iterator{
    public abstract boolean has_next();
    public abstract Object get_next();
}
```



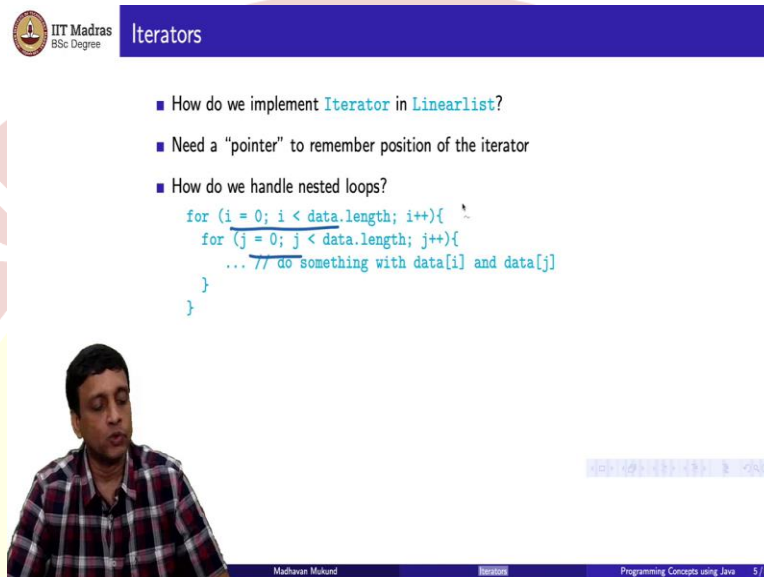
So, what we need is a kind of abstraction of what it means to iterate through a list. So, what it means to iterate through a list is I want to start from the first element and keep going until I run out of elements. So, I need to start at the beginning of the list. And therefore, I am at the first element. And then so long as there is a next element, I will get something, get it and do something with it. So, I will keep doing this. So, it is a bit like walking down a linked list concretely, but it is a more abstract thing.

So, as long as there is a next element in a sequence, get that next element, process it in whatever way you want. For example, if you are computing the sum of the list, you will add it to a running total. If you want to check whether it is of a valid type and then do something you can do that. And then once you are done with it, you go to the next one and you keep doing this so long is, once you come to an element which says there is no next element then you stop. So, this is what we need to capture and the way we have been doing this kind of capability capturing is through interfaces.

So, we can define an interface called an `Iterator`, which has these two functions, the first thing is something called `has_next()`. It tells me whether or not there is one more element. So, for instance, if I have an empty list, then right at the beginning the iterator will say `has_next()` is false, because there is no first element. So, even right at the beginning, so starting at the beginning of the list itself will require me to check whether `has_next()` is true or not. And as long as `has_next()` is true, I can do this `get_next()` and get an object so that is the idea.

So, we have these two functions, one which tells me whether I am at the end of the list, and the other one which tells me, gets me the next value in the list if I am not at the end, but then remember, it must be keeping track of where I am in the list.

(Refer Slide Time: 05:47)



IIT Madras
BSc Degree

Iterators

- How do we implement `Iterator` in `Linearlist`?
- Need a "pointer" to remember position of the iterator
- How do we handle nested loops?

```
for (i = 0; i < data.length; i++){  
    for (j = 0; j < data.length; j++){  
        ... // do something with data[i] and data[j]  
    }  
}
```

Madhavan Mukund

Programming Concepts using Java 5/8

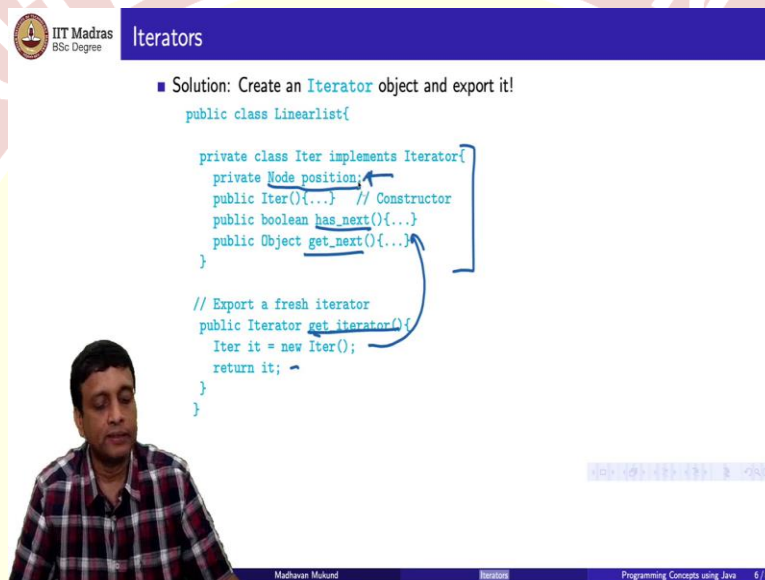
So, if I am going to implement this iterator interface inside a concrete linear list like the one that we described before, we need to maintain inside this iterator a pointer to where it is. So, initially, is the first element and then it, so the iterator must maintain, just like as we had for that second version of that query object, so the query object has a function, but it also maintained an instance variable as so the number of queries that have been made. So, that was not really directly reflected in the query itself. The query did not ask how many queries are there. But as a bookkeeping measure, it was keeping track of the number of queries that have been made so that it can cut off that query object after a certain limit.

In the same way this `has_next()` and `get_next()` do not explicitly reference the position, but obviously the iterator must remember the position. So, the object that implements this linear list iterator must remember where it is so that it can use that to tell you whether there is a next or not. The other question is nested loops. So, we do not typically need only one iterator, we need multiple iterators, which run independent of each other.

So, when we do a nested loop like this, it says for $i = 0$, $i < \text{something}$, do something $i++$ and so while i is at a fixed value, j is iterating. So, when j is iterating, i is fixed. And then when i, j gets

reset to 0 when i increments. So, these are two separate processes. So, there is a has_next() and I get_next() is happening in j, there is a has_next() and get_next() is happening in i, and they do not interfere with each other. So, we would need to be able to have this kind of capability also in our iterators. So, the iterator should support not just one iteration through my collection, but it should support these parallel independent iterations which I can nest and go through in some complicated way if I want to compare all pairs of elements and so on.

(Refer Slide Time: 07:45)



Iterators

■ Solution: Create an `Iterator` object and export it!

```
public class Linearlist{

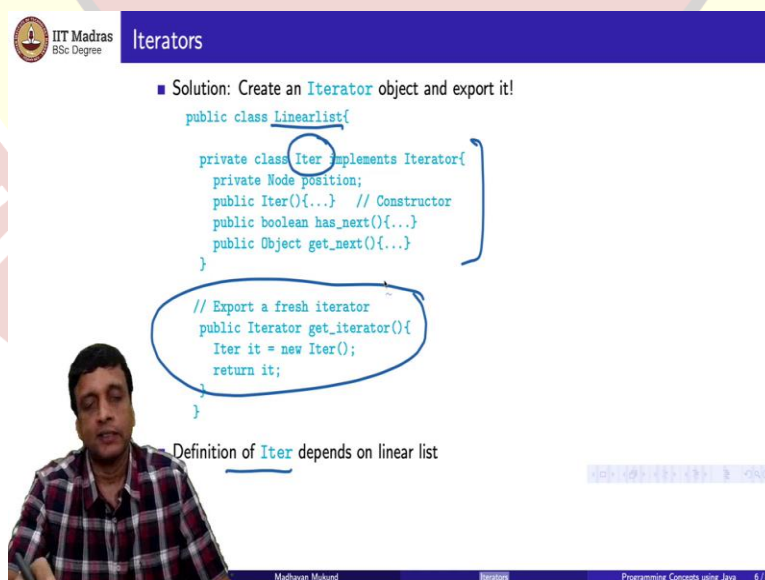
    private class Iter implements Iterator{
        private Node position;
        public Iter(){...} // Constructor
        public boolean has_next(){...}
        public Object get_next(){...}
    }

    // Export a fresh iterator
    public Iterator get_iterator(){
        Iter it = new Iter();
        return it;
    }
}
```

Madhavan Mukund

Iterators

Programming Concepts using Java 6/8



Iterators

■ Solution: Create an `Iterator` object and export it!

```
public class Linearlist{

    private class Iter implements Iterator{
        private Node position;
        public Iter(){...} // Constructor
        public boolean has_next(){...}
        public Object get_next(){...}
    }

    // Export a fresh iterator
    public Iterator get_iterator(){
        Iter it = new Iter();
        return it;
    }
}
```

Definition of `Iter` depends on linear list

Madhavan Mukund

Iterators

Programming Concepts using Java 6/8

So, as we would expect, the solution to all these things is to create an appropriate iterator object and then export it with the interface which allows me to query has_next() and obtain get_next().

So, if I use my node implementation, for example, of linear list, then I might remember where I am. So, this is now my implementation of iterator. So, I have a private class as before. So, I will create an object of this type and return it. So, if you say `get_iterator()`, I will create a new object of this type and return it. So, that is the behavior of my iterator function. So, I will return a new iterator.

So, in order to return a new iterator, I need to create it, but then that is a private class. But inside that iterator, I need to implement the thing. So, I need to have an implementation of `has_next()` and `get_next()`. And this `has_next()` and `get_next()` will depend on the position. So, the actual implementation, so the actual definition of this class will vary according to how the linear list is actually implemented.

If it was implemented as an array, then I will just maintain the position as an index in the array, so I will start with 0, and `has_next()` will just be whether or not the current index is within the limit of the length of the array, and if so `get_next()` will just increment this index. In the case of linked lists, this thing will tell me whether or not there is a next node in the list. And if there is a next node then `has_next()` will go from this node to the value `m.next`.

So, depending on how we actually implement the actual linear list, this class implementation will change, but this part will more or less be the same. If somebody asked me for an iterator, I will just return it after constructing it and the constructor will automatically take care of the type of iterator that I need.

(Refer Slide Time: 09:43)

IIT Madras BSc Degree

Iterators

- Now, we can traverse the list externally as follows:

```
Linearlist l = new Linearlist();
Object o;
Iterator i = l.get_iterator();
while (i.has_next()){
    o = i.get_next();
    ... // do something with o
}
```

Handwritten notes: A bracket groups the first three lines. An arrow points from `o = i.get_next();` to the word "False" with a line through it, indicating the loop condition. Another arrow points from the `while` loop to the `o` variable.

Madhavan Mukund

Iterators

Programming Concepts using Java 7/8

IIT Madras BSc Degree

Iterators

- Now, we can traverse the list externally as follows:
- For nested loops, acquire multiple iterators!

```
Linearlist l = new Linearlist();
...
Object o;
Iterator i = l.get_iterator();

while (i.has_next()){
    o = i.get_next();
    ... // do something with o
}

...

Linearlist l = new Linearlist();
...
Object oi, oj;
Iterator i, j;

i = l.get_iterator();
while (i.has_next()){
    oi = i.get_next();
    j = l.get_iterator();
    while (j.has_next()){
        oj = j.get_next();
        ... // do something with oi, oj
    }
}
```

Handwritten notes: A bracket groups the first `while` loop. An arrow points from the `oi` variable to the `oj` variable in the nested loop.

Madhavan Mukund

Iterators

Programming Concepts using Java 7/8

So, if I want to use this iterator, how do I do it? Well, I asked for an iterator. So, I have a linear list `l`, which I have created. And then over a period of time I have kind of created and populated with some values and now I want to iterate over the list. So, I need two things. I need an iterator and I need something to hold the value that the iterator gives me. So, I create a variable `o` of type object which can hold each individual value that I need to examine and I create a new iterator by using this `get iterator` function. So, and the `get iterator` function is called, implicitly, this iterator is at the beginning.

But remember that if it is empty also, I will get an iterator. So, the key is that I should always first check whether there is a next element. So, I do not start automatically the 0th element. I start with like an initialized iterator which is not pointing to any element. So, when I say `has_next()`, if there is at least one element, then this will return true. And once I know there is at least one element, I can get the first element, and I can store it in this variable `o` which I created. So, I store this value in `o` and I do something with it, whatever I need to do with that thing.

And then I come back and I check, I have moved one position down. So, inside the iterator, remember, it is remaining, it is remembering where I am. So, it is now pointing to the next index in the array or it is pointing to the next node in the list. So, now I again say `has_next()`. So, `has_next` internally depending on the representation will either say, yes, there is one more, or will say, no, there is no more. So, if there is one more, I will get the next element, I will increment by one, and so on. And I will keep doing this until I reach the end of the linear list. At the end of the linear list this `has_next()` will be false. And once it is false, I will come up.

So, you use it essentially in the same structure as a normal while loop except you use instead of incrementing the index $i = i + 1$ and checking the index against the length of an array explicitly, you do it indirectly by using this `has_next()` and `get_next()`. So, they, this interface of iterator abstractly captures the capability of proceeding to the next element or indicating that this the list is over.

So, what about nested loops now? Well, if you want nested loops, you need two iterators. So, you will keep track of two iterators and each iterator will, so remember we are trying to say, for example, check whether there are two values in the list which are the same, so for each position `i` and for each position `j`, you want to compare the object with position `i` with the, compare object with position `j`, so you need two variables to hold these objects. So, you will have an `oi` and an `oj` and we will have two iterators, `i` and `j`.

So, initially, we start off by initializing the outer iterator and so long as the outer iterator has a next, so this is an outer loop on `i`, we will get that next one, and now we will initialize the inner iterator. So, now the `i` is pointing to the first element in the list and now we say `j` pointing to the 0th or the beginning of the list and now `j` will run through its own loop and it will go from beginning to end. So, for the fixed `i` you will run through all the elements in `j`, this is exactly a

nested loop. And you will have oi coming from that outer iteration and oj will keep changing in the inner iteration. So, you will now compare oi and oj, do whatever you want to do with oi and oj, go to the next oj and so on until you exhaust all the js.

When the j's are over this loop here will terminate. So, you will come back here. And you will check whether i can go ahead one more step. If so, I will progress by i dot get next and then you start a new j loop. So, the nested loop is exactly the same in this iterator mode as it would be in a conventional thing. Only thing is you have to keep track of the syntax a little bit by using this has_next() and get_next().

(Refer Slide Time: 13:34)

Summary

- Iterators are another example of interaction with state
 - Each iterator needs to remember its position in the list
- Export an object with a prespecified interface to handle the interaction
- The new Java `for` over lists implicitly constructs and uses an iterator

```
for (Type x : a)
do something with x;
}
```

```
for(i=0; i<n; i++){
}
```

Madhavan Mukund | Iteration | Programming Concepts using Java 8/8

So, as you can see an iterator is again an interaction with the state. We want, so the iterator has access to the linear list. It can get you the value at some position in the list by remembering where it is. So, it is always going to get to the current position in the list and return it to you or current + 1 if you think of get_next(). And it is also going to tell you whether we reached the end of the list or not.

So, it maintains the state of where in the earlier case when we looked at this query of the database, we were maintaining the state of the object making the queries. So, here the object making the queries is at a certain position in the list. So, your iterator is at a certain position, so the iterator is keeping track of where you are in the list.

So, as before, the key to doing this kind of interaction with state is that you have an interface which is pre-specified and you export a private object which implements this interface. So, from the outside point of view, you know exactly what to do, you know you have to use `get_next()` and `has_next()`. You have no idea how this `get_next()` and `has_next()` actually implemented because it is a private concern. And because the class came from inside the linear list class, it has access to the private data of the linear list class. So, it can report the relevant information to you outside.

Now, if you remember in the first week, when we looked at the control flow operators in Java, we said that, in addition to that for loop with this `for i=0, i<n, i++` this kind of loop, which is the conventional C type of loop, which takes you through a range of indices, we said that there is a Python type of loop also, which says that you can say, `for x : a`. So, `a`, for example, is an array, only thing as we said that you must use a locally declared `x`, otherwise this is a compiler error. But this essentially allows you to iterate over the objects of an array directly.

Now, an array is, of course, a built-in type, but we will see later on that there are other linear classes. So, there are built-in classes in Java, which are like this linear list that we were talking about. So, this for loop actually works for all these kinds of linear classes. So, in a way, although iterators were a fundamental requirement in any object oriented setting where you keep abstract linear lists by providing this more functional for Java has kind of moved ahead so that you do not really have to use an iterator explicitly in many cases, you can just use this more Python like for loop, but it is important to know that iterators as a concept require you to have the same kind of interaction with state as many other things so you can implement it using a combination of private classes and interfaces.